Types, UTF-8, Iota et Controle

Objectifs

1. Exploiter les *Zero Values*, le typage strict et les conversions explicites en Go.
2. Manipuler les chaînes de caractères encodées en UTF-8 en distinguant les octets (`byte`) des points de code Unicode (`rune`).
3. Modéliser des énumérations et des masques binaires (*Bitflags*) à l'aide du générateur `iota`.
4. Maîtriser les constructions idiomatiques de contrôle (`if` à portée locale, les 4 formes de `for`, et le `switch` sans condition).

Exercice 1 : Analyseur et manipulateur de chaînes UTF-8

En C, une chaîne est un tableau d'octets terminé par `\0`. En Go, une `string` est une séquence immuable d'octets UTF-8 où un caractère humain peut occuper entre 1 et 4 octets.

1. Écrivez un programme qui analyse la chaîne suivante :

```
Go
phrase := "Go : Sûr & Rapide à 100% !"
```

Parcours par octets (byte) vs parcours par caractères (rune) :

   - Affichez la longueur brute en octets retournée par `len(phrase)`.
   - À l'aide d'une boucle `for` classique (`for i := 0; i < len(phrase); i++`), comptez le nombre d'octets et observez ce qui s'affiche si vous tentez d'imprimer `phrase[i]` sous forme de caractère (`%c`).
   - À l'aide d'une boucle `for ... range` sur `phrase`, comptez le nombre réel de caractères Unicode (`rune`) et le nombre exact de caractères dits "multi-octets" (ceux dont le point de code Unicode est supérieur à `127`, c'est-à-dire hors table ASCII standard).
2. **Censure et masquage UTF-8 :**
   - Comme une `string` est immuable, vous ne pouvez pas modifier un caractère directement par son index.
   - Écrivez une boucle qui construit une nouvelle chaîne à partir de `phrase` en remplaçant tous les chiffres (`'0'` à `'9'`) par le caractère `'*'`, et en supprimant

tous les caractères non-ASCII (comme les accents et les émojis), en utilisant un `switch` sans condition sur chaque `rune`.

Exercice 2 : Modélisation de permissions système avec `iota`

Vous développez le moteur de contrôle d'accès d'un orchestrateur de conteneurs. Au lieu d'utiliser plusieurs booléens coûteux en mémoire, vous devez compacter les droits d'un utilisateur dans un seul octet (`uint8`) grâce aux opérateurs binaires.

1. Déclarez un type strict `Permission uint8` et un type `Role uint8`.
2. Utilisez un premier bloc `const` avec `iota` pour définir les rôles suivants :
    - `RoleAucun` (doit valoir `0`, la *Zero Value* par défaut)
    - `RoleLecteur` (`1`)
    - `RoleEditeur` (`2`)
    - `RoleAdmin` (`3`)
3. Utilisez un second bloc `const` avec l'expression `1 << iota` pour générer les masques binaires suivants de type `Permission` :
    - `PermLecture` (`1`, soit `00000001` en binaire)
    - `PermEcriture` (`2`, soit `00000010`)
    - `PermExecution` (`4`, soit `00000100`)
    - `PermSuppression` (`8`, soit `00001000`)
    - `PermReseau` (`16`, soit `00010000`)
4. À l'aide d'un `switch` sur le `Role`, attribuez automatiquement le masque de permissions correspondant à un profil :
    - `RoleLecteur` : Lecture uniquement.
    - `RoleEditeur` : Lecture + Écriture + Exécution (combinés avec le OU binaire `|`).
    - `RoleAdmin` : Toutes les permissions.
5. Affichez le masque obtenu en binaire sur 8 bits (verbe `fmt.Printf("%08b\n", masque)`), puis testez la présence d'un droit précis avec l'opérateur ET binaire (`&`) et le retrait d'un droit avec l'opérateur ET-NON spécifique à Go (`&^`, appelé *Bit Clear*).

Exercice 3 : Boucles avancées, Labels et `if` à portée locale

Vous devez écrire l'algorithme de contrôle d'un scanner de ports simulé sur une matrice d'adresses IP et de ports réseau.

1. Initialisez une boucle externe qui parcourt 3 sous-réseaux simulés (`idReseau` de `1` à `3`) et une boucle interne qui parcourt les ports de `80` à `90`.
2. Utilisez un **label de boucle** (`ScanGlobal:`) placé juste au-dessus de la boucle externe :

- Si le couple `(idReseau == 2 && port == 83)` est rencontré, considérez qu'il s'agit d'un port réservé : ignorez immédiatement le reste des ports du réseau `2` et passez directement au réseau `3` (`continue ScanGlobal`).
- Si le couple `(idReseau == 3 && port == 85)` est rencontré, simulez une alerte d'intrusion critique qui doit **interrompre instantanément** les deux boucles imbriquées (`break ScanGlobal`).

3. Dans le corps de la boucle, utilisez la syntaxe du **`if` avec déclaration d'initialisation locale** (`if latence := (idReseau * 10) + (port % 7); latence > 25 { ... }`) pour afficher si le port répond rapidement ou lentement, tout en garantissant que la variable `latence` n'existe pas en dehors du bloc `if / else`.